WEEKLY MINI PROJECT 03 · TEAM 4

Who Doesn't Know?

Team Work Split · Overview

6 work packages | Own api + mocks per person | Nobody waits

PROJECT	TEAM LEADER	TEAM MEMBERS
Who Doesn't Know?	**Taima Watfeh**	**Ali Al Hamwi, Ali Mikdad, Alaa Ismaiel, Nawar Hasan, Yazan Shartouh**

BACKEND	FRONTEND	LEADER
Ali Al Hamwi, Ali Mikdad	**Yazan, Nawar, Alaa**	**Taima: foundation, integration, demo**

Every person owns their own files: their own endpoints, their own API file, their own mock data. Nobody waits.

What you already know from class: the team is reusing the same NestJS pattern from the current backend: Module → Controller → Service, DTO validation, Mongoose schemas, Swagger decorators, seed-on-empty, readable HTTP exceptions, the global exception filter, Logger, and InternalToolGuard. The new game modules follow that pattern; nobody starts from zero.

1 Who Does What

Member	Builds	Own API file	Own mock file
Taima · Leader	Day-0 foundation, README, integration, demo, slides	`http.ts` (shared helper)	none
Ali Al Hamwi · Backend	GamesModule + TipsModule: controllers, services, DTOs, schemas and seed	`backend/src/games`, `tips`	`scripts/seed-tips.ts`
Ali Mikdad · Backend	GameplayModule: controller, service, DTOs, gameplay schemas and rules	`backend/src/gameplay`	`scripts/seed-dev-game.ts`
Yazan · Frontend	Session, polling hook, routing, error banner, Home, Lobby	`lobby.api.ts`, `state.api.ts`	`lobby.mock.ts`, `state.mock.ts`
Nawar · Frontend	Game screen, Voting screen	`game.api.ts`, `voting.api.ts`	`game.mock.ts`, `voting.mock.ts`
Alaa · Frontend	Round Decision screen, Result screen	`decision.api.ts`	`decision.mock.ts`, `result.mock.ts`

Frontend structure: follow the previous project style: only `pages/` for screens and `components/` for reusable UI, plus the allowed `api/` and `mocks/` folders, with exactly one shared `src/types.ts` file at the src root.

2 Frontend: Mock Data First

Each frontend member writes the API functions for their own screens and a mock file with fake data for them. A single flag switches every api file from mock to real.

PHASE 1 · DAYS 1–3

Mocks ON

`VITE_USE_MOCKS=true`. Your api file returns data from **your own** mock file.

PHASE 2 · DAY 4

Mocks OFF

`VITE_USE_MOCKS=false`. The same api functions call the real backend.

PHASE 3 · DAY 4–5

Fix together

Run the flows on the real backend; fix any difference in your own files.

3 Frozen API Contract

Method	Path	Body	Purpose	Owner
POST	`/games`	`{ displayName }`	Create game, caller becomes host	Ali Al Hamwi
POST	`/games/join`	`{ roomCode, displayName }`	Join a game in lobby	Ali Al Hamwi
POST	`/games/:roomCode/start`	`{ playerId }`	Host starts the game	Ali Al Hamwi
GET	`/games/:roomCode/state`	none	**Public** game state (polled)	Ali Al Hamwi
GET	`/games/:roomCode/me?playerId=`	query	**Player-specific** role view	Ali Al Hamwi
POST	`/games/:roomCode/questions`	`{ playerId, targetRef, text }`	Current player asks	Ali Mikdad
POST	`/games/:roomCode/questions/:questionId/answer`	`{ playerId, text }`	Target answers	Ali Mikdad
POST	`/games/:roomCode/decisions`	`{ playerId, choice }`	`choice`: `continue` | `vote`	Ali Mikdad
POST	`/games/:roomCode/votes`	`{ playerId, suspectRef }`	Suspect vote	Ali Mikdad
GET / POST / PATCH	`/tips`, `/tips/:id`, `/tips/:id/deactivate`	`{ text, category }`	**Internal only**, InternalToolGuard header	Ali Al Hamwi

```
// Response of POST /games and POST /games/join
{ gameId: number, roomCode: string, playerId: string /* private, e.g. p_a82f31 */, playerRef: number /*
public */ }
```

4 Frozen Data Model

Separate collections linked by `gameId`; the public state is built field by field, so secret data is never copied into it.

Collection	Fields	Indexes
`games`	`id, roomCode, status, roundNumber, hostRef, hiddenPlayerRef, secretTipId, turnOrder[], currentTurnIndex, turnsCompletedInRound, pendingQuestionId, decisionNumber, votingNumber, lastDecisionOutcome, lastVoteTie, winner, accusedRef, createdAt`	`roomCode` (+ `status`)
`players`	`id` (= public ref), `gameId, displayName, normalizedName, tempId, isHost, joinedAt`	`tempId` unique; `{gameId, normalizedName}` unique; `gameId`
`questions`	`id, gameId, round, askerRef, targetRef, text, answer, askedAt, answeredAt`	`{gameId, id}`
`round_decisions`	`id, gameId, decisionNumber, playerRef, choice`	`{gameId, decisionNumber, playerRef}` unique
`suspect_votes`	`id, gameId, votingNumber, voterRef, suspectRef`	`{gameId, votingNumber, voterRef}` unique
`secret_tips`	`id, text, category, active`	`active`
`counters`	`name, seq` (numeric id generator, as in B4F Board)	`name` unique

5 Timeline

When	Goal
Day 0	Taima pushes the foundation (≈2 h). Everyone creates their branch.
Days 1–3	Everyone builds alone: frontend on mocks, backend on Swagger and seed scripts.
Day 4	Merge into `develop`, switch mocks off, run Flows A–E together.
Day 5	Fixes, README, rehearsal with 4+ devices.

6 Git Rules

- Branches: `main`, `develop`, `feature/<name>-<topic>`.
- Commit only inside your own files (listed in your package). PRs into `develop` reviewed by Taima.

- Contract changes only through Taima, announced to everyone the same day.